



Sécurité des Infrastructures Web

RAPPORT DE PROJET DE FIN D'ÉTUDES

Sujet : Mitigating DDoS and Injection Attacks

Membres du groupe :

HMAIDI Ilef
SAHLI Sarra
AKAICHI Mohamed Rayen

Classe : CY Tech – ING3 Cybersécurité (Groupe A)

Année universitaire : 2025/2026

Table des matières

Liste des Acronymes	5
Introduction Générale	6
1 Objectifs de Sécurité et Analyse des Menaces	8
1.1 Contexte et problématique	8
1.2 Objectifs du projet	8
1.3 Rôle du Reverse Proxy dans l'architecture	9
1.4 Analyse des attaques par injection	9
1.4.1 Injection SQL (SQLi)	9
1.4.2 Cross-Site Scripting (XSS)	9
1.4.3 XML External Entity (XXE)	9
1.4.4 Attaques DDoS aux couches 3 et 4	10
1.4.5 Attaques DDoS à la couche 7	10
1.5 Classification des menaces	10
1.6 Solutions existantes	10
1.7 Solution proposée : Architecture de défense multicouche	11
1.7.1 Composants techniques de la solution	11
1.7.2 Mécanisme de remédiation automatique	12
1.7.3 Infrastructure Backend	12
2 Déploiement de l'Environnement de Test (Proof of Concept)	13
2.1 Introduction	13
2.2 Topologie de l'infrastructure réseau et virtuelle	13
2.2.1 La machine attaquante (Kali Linux)	13
2.2.2 La machine cible et défensive (Ubuntu Server)	14
2.3 Déploiement conteneurisé de l'application et du Reverse Proxy	14
2.4 Implémentation des couches de défense	14
2.4.1 Filtrage réseau bas niveau (iptables)	14
2.4.2 Intégration du WAF ModSecurity	14
2.4.3 Déploiement de l'IPS comportemental CrowdSec	15
3 Durcissement du Réseau et Attaques Volumétriques (Couche 4)	16
3.1 Introduction	16
3.2 Contexte et Analyse de la Menace : Le SYN Flood	16
3.3 Implémentation des Défenses Réseau	17
3.3.1 Durcissement du Noyau Linux (Sysctl)	17
3.3.2 Limitation de Débit Dynamique (Iptables Hashlimit)	17
3.4 Validation et Preuves de Concept	18

3.5	Conclusion	19
4	Protection Applicative et Volumétrie (Couche 7)	20
4.1	Introduction	20
4.2	Mitigation Volumétrie : L'Attaque Slowloris	20
4.2.1	Contexte et Analyse de la Menace	20
4.2.2	Durcissement de Nginx (Timeouts et Rate Limiting)	21
4.3	Filtrage Applicatif : WAF ModSecurity	22
4.3.1	Principes d'Inspection Profonde	22
4.3.2	Neutralisation des Injections (SQLi et XSS)	22
4.3.3	Restriction d'accès aux pages sensibles	23
4.4	Résultats et Preuves de Concept	23
4.5	Conclusion	25
5	Réponse Active (IPS) et Supervision	26
5.1	Introduction	26
5.2	Contexte et Enjeux de la Sécurité Active	26
5.3	Déploiement de l'IPS : CrowdSec	26
5.3.1	Architecture	26
5.3.2	Mécanisme de Détection (Leaky Bucket)	27
5.3.3	Remédiation Réseau via IPSet	27
5.4	Supervision et Pilotage (SOC) avec Metabase	27
5.5	Tests et Validation de l'IPS	28
5.6	Conclusion	29
	Conclusion Générale	30
	Annexes Techniques	31
A.1	Scripts de Durcissement Réseau (Couche 4)	31
A.1.1	Configuration Sysctl (Protection Mémoire Kernel)	31
A.1.2	Script Iptables (Limitation de débit dynamique)	31
A.2	Configurations de Sécurité Applicative (Couche 7)	32
A.2.1	Durcissement Nginx (Anti-Slowloris et Rate Limiting)	32
A.2.2	Activation du WAF ModSecurity sur le Virtual Host	32
A.3	Configurations de la Réponse Active et Supervision (Couche IPS)	32
A.3.1	Déploiement et liaison de CrowdSec avec le WAF	32
A.3.2	Déploiement de Metabase (Docker Compose)	33
A.3.3	Requêtes SQL Analytiques (Dashboard SOC)	33
	Bibliographie	35

Table des figures

1.1	Architecture de défense proposée intégrant Nginx, ModSecurity et CrowdSec	11
3.1	Mécanisme d'épuisement des ressources via SYN Flood	17
3.2	Interception en temps réel des paquets hostiles par Iptables	18
4.1	Mécanisme de saturation des ressources applicatives par Slowloris	21
4.2	Échec de l'attaque Slowloris face à l'architecture durcie de Nginx	22
4.3	Échec de l'accès à la page setup.php	23
4.4	Interception immédiate d'une charge utile malveillante par le WAF	24
5.1	Architecture Agent/Bouncer et flux de décision de CrowdSec	27
5.2	Tableau de bord SOC pour la supervision en temps réel	28
5.3	Constat du bannissement effectif de l'attaquant au niveau du pare-feu . . .	28

Liste des tableaux

1.1	Classification des menaces prioritaires	10
1.2	Comparatif détaillé des solutions de sécurité existantes	11
3.1	Comparatif des performances sous attaque volumétrique	18
4.1	Bilan des interceptions applicatives par ModSecurity	24
5.1	Synthèse des réponses actives du système CrowdSec	29

Liste des Acronymes

CRS	Core Rule Set
DDoS	Distributed Denial of Service (Déni de Service Distribué)
HTTP	HyperText Transfer Protocol
IDS	Intrusion Detection System (Système de Détection d’Intrusion)
IPS	Intrusion Prevention System (Système de Prévention d’Intrusion)
NAT	Network Address Translation
OSI	Open Systems Interconnection
OWASP	Open Worldwide Application Security Project
PoC	Proof of Concept (Preuve de Concept)
SOC	Security Operations Center (Centre Opérationnel de Sécurité)
SQLi	SQL Injection (Injection SQL)
TCP	Transmission Control Protocol
TLS/SSL	Transport Layer Security / Secure Sockets Layer
VM	Virtual Machine (Machine Virtuelle)
WAF	Web Application Firewall (Pare-feu Applicatif Web)
XSS	Cross-Site Scripting
XXE	XML External Entity

Introduction Générale

L'évolution rapide des technologies web et la dépendance croissante aux services en ligne ont profondément transformé le fonctionnement des organisations modernes. La cybersécurité joue désormais un rôle vital pour garantir la disponibilité, l'intégrité et la confidentialité des données face à des menaces cybernétiques de plus en plus sophistiquées et automatisées. Aujourd'hui, les infrastructures web ne doivent plus seulement être performantes et fonctionnelles ; elles doivent être intrinsèquement résilientes et capables de repousser des intrusions externes à différents niveaux du modèle OSI.

Dans ce paysage numérique hostile, les organisations sont souvent confrontées au défi de sécuriser des applications web vulnérables, parfois vieillissantes (legacy), sans avoir la possibilité d'en modifier le code source. Face à la recrudescence des attaques par déni de service (DDoS) et à l'exploitation massive des failles applicatives, il est devenu impératif de dépasser le modèle du pare-feu traditionnel pour adopter une posture de défense proactive et multicouche, capable de s'adapter dynamiquement aux nouveaux risques.

Dans ce contexte, le projet sur lequel ce rapport est basé se concentre sur la conception, le déploiement et la validation d'une architecture de sécurité en profondeur (*Defense in Depth*). Le système proposé vise à doter une infrastructure web vulnérable d'un bouclier protecteur robuste, intégrant les meilleures pratiques de sécurité actuelles. Cela inclut le durcissement du noyau réseau, la mise en place d'un pare-feu applicatif (WAF) pour contrer les vulnérabilités du TOP 10 OWASP, ainsi que le déploiement d'un système de prévention d'intrusion (IPS) basé sur l'analyse comportementale.

L'objectif de ce projet est de fournir une solution centralisée, automatisée et hautement sécurisée permettant de neutraliser les attaques avant qu'elles n'atteignent les serveurs backend. Ce système intègre également une dimension de supervision via un tableau de bord, offrant aux administrateurs une visibilité en temps réel sur les menaces, tout en garantissant une continuité de service transparente pour les utilisateurs légitimes.

Le présent rapport est divisé en cinq chapitres principaux. Dans le premier chapitre, intitulé "Objectifs de Sécurité et Analyse des Menaces", nous présentons le contexte du projet, la classification des cyberattaques ciblées et l'architecture globale proposée. Dans le deuxième chapitre, "Déploiement de l'Environnement de Laboratoire", nous détaillons la topologie de l'infrastructure de test (Proof of Concept) et la conteneurisation des services. Le troisième chapitre, "Durcissement Réseau", aborde la protection de la couche 4 contre les attaques volumétriques de type SYN Flood. Dans le quatrième chapitre, "Sécurité Applicative", nous présentons la configuration du Reverse Proxy et du WAF pour bloquer les attaques de couche 7 (Slowloris, Injections SQL, XSS). Enfin, dans le cinquième chapitre, "Réponse Active et Supervision", nous détaillons l'implémentation de

l'IPS CrowdSec et la création du tableau de bord d'analyse des menaces. Nous concluons ce rapport en dressant le bilan des résultats obtenus et en proposant des perspectives d'évolution pour le projet.

Chapitre 1

Objectifs de Sécurité et Analyse des Menaces

1.1 Contexte et problématique

Avec la digitalisation croissante des services, les applications web sont devenues des cibles privilégiées pour les cyberattaques. Les entreprises dépendent fortement de la disponibilité, de l'intégrité et de la confidentialité de leurs services en ligne. Cependant, ces applications sont souvent exposées à Internet, ce qui les rend vulnérables à différentes formes d'attaques.

Parmi les menaces les plus critiques figurent :

- Les attaques par déni de service (DDoS), qui visent à rendre un service indisponible.
- Les attaques par injection, qui exploitent des failles applicatives afin de voler ou de manipuler des données sensibles.

Dans ce contexte, la mise en place d'une architecture de sécurité robuste devient indispensable pour protéger les serveurs web contre ces menaces.

1.2 Objectifs du projet

L'objectif principal de ce projet est de concevoir et mettre en œuvre une architecture de défense efficace pour des serveurs web placés derrière un Reverse Proxy. Les objectifs spécifiques sont :

- Identifier et analyser les principales menaces ciblant les applications web.
- Étudier les attaques DDoS aux différentes couches du modèle OSI (couches 3, 4 et 7).
- Étudier les attaques par injection telles que SQLi, XSS et XXE.
- Mettre en place des mécanismes de protection adaptés comme le WAF ModSecurity et l'IDS/IPS CrowdSec.
- Évaluer l'efficacité des solutions via des tests "Avant/Après" sur une application vulnérable (DVWA).

1.3 Rôle du Reverse Proxy dans l'architecture

Le Reverse Proxy constitue un élément central de l'architecture de sécurité proposée. Il agit comme un intermédiaire entre les clients externes et les serveurs backend.

Ses principaux rôles sont :

- **Masquage d'identité** : Il masque l'identité des serveurs backend, réduisant ainsi leur exposition directe.
- **Gestion du chiffrement SSL/TLS** : Il assure la terminaison SSL, ce qui est indispensable pour inspecter le trafic entrant.
- **Filtrage du trafic** : Il sert de point d'entrée unique pour contrôler le trafic avant qu'il n'atteigne les serveurs internes.

1.4 Analyse des attaques par injection

Les attaques par injection exploitent une mauvaise validation des données entrantes fournies par les utilisateurs.

1.4.1 Injection SQL (SQLi)

L'injection SQL est l'une des menaces les plus critiques pour la confidentialité des données. Elle survient lorsqu'un attaquant insère du code SQL malveillant dans les champs d'entrée d'une application, tels que les formulaires de saisie, les cookies ou les paramètres d'URL.

Mécanisme : L'attaquant manipule la requête SQL originale pour forcer le serveur de base de données à exécuter des commandes non autorisées.

Impacts principaux : Cette faille permet un accès non autorisé à la base de données, facilitant le vol, la modification ou la suppression de données sensibles, ainsi que la compromission totale de l'application.

Sévérité : Cette menace est classée comme critique en raison de son impact direct sur la confidentialité des systèmes.

1.4.2 Cross-Site Scripting (XSS)

Contrairement à la SQLi qui cible la base de données, le XSS vise directement les utilisateurs finaux de l'application.

Mécanisme : L'attaquant injecte des scripts malveillants (généralement du JavaScript) qui s'exécutent au sein du navigateur de la victime lors de la consultation d'une page légitime.

Impacts principaux : Cette vulnérabilité permet le vol de jetons de session (cookies), la redirection vers des sites malveillants, ou l'exécution d'actions au nom de l'utilisateur.

Sévérité : Le niveau de sévérité est jugé critique car il compromet l'intégrité de l'expérience utilisateur et la confidentialité des sessions.

1.4.3 XML External Entity (XXE)

L'attaque par entité externe XML (XXE) cible les applications qui traitent et analysent des fichiers ou des flux de données au format XML.

Vecteurs d'attaque : Cette faille est souvent exploitée via l'upload de fichiers XML malveillants ou l'interception de flux SOAP.

Impacts principaux : Elle peut entraîner la lecture de fichiers sensibles présents sur le serveur, l'exfiltration de données internes ou provoquer un déni de service.

Sévérité : L'impact combiné sur la confidentialité et la disponibilité rend cette menace critique.

1.4.4 Attaques DDoS aux couches 3 et 4

Ces attaques ciblent principalement le réseau et le transport :

- **ICMP Flood** : Saturation via des requêtes ICMP Echo.
- **SYN Flood** : Envoi massif de paquets TCP SYN sans finaliser la connexion.
- **UDP Flood** : Envoi de paquets UDP vers des ports aléatoires.
- **Amplification et réflexion** : Utilisation de serveurs tiers (DNS, NTP) pour amplifier le volume de l'attaque.

1.4.5 Attaques DDoS à la couche 7

Celles-ci ciblent les ressources applicatives et sont plus difficiles à détecter :

- **HTTP Flood** : Submersion par un grand nombre de requêtes HTTP GET ou POST légitimes en apparence.
- **Slowloris** : Maintien de connexions TCP ouvertes par l'envoi d'en-têtes HTTP incomplets et très lents.
- **DDoS basé sur les ressources** : Ciblage d'URL nécessitant une exécution complexe (recherches en base de données).

1.5 Classification des menaces

Le tableau suivant synthétise la classification des menaces identifiées :

Menace	Vecteur d'attaque	Impact	Sévérité
DDoS	Saturation réseau / ressources	Disponibilité	Critique
SQLI	Paramètres URL, Formulaires	Confidentialité	Critique
XXE	Upload fichiers XML, Flux SOAP	Conf. & Dispo.	Critique
XSS	Scripts navigateur client	Confidentialité	Critique

TABLE 1.1 – Classification des menaces prioritaires

1.6 Solutions existantes

L'analyse des solutions actuelles permet de comparer les différentes approches technologiques pour la protection des infrastructures web :

Solution	Type	Couches	Forces principales	Limites typiques
Cloudflare	Cloud Anycast	3, 4 et 7	Le serveur web ne reçoit que le trafic filtré. Règles managées.	Dépendance au fournisseur. Partage de certificats (SSL). Coût de 20€ à 250€.
Nginx avec ModSecurity + CrowdSec	Open-Source	3, 4 et 7	Coût logiciel nul. Très flexible. Inspection HTTP profonde. Pas de dépendance à un tiers.	Configuration complexe. Tuning long pour réduire les faux positifs. Consommation CPU/RAM.
F5 BIG-IP	Appliance / VM	3, 4 et 7	WAF avancé. Basé sur l'analyse comportementale et le machine learning.	Coût élevé. Entre 50 000€ et 120 000€ selon la gamme.

TABLE 1.2 – Comparatif détaillé des solutions de sécurité existantes

1.7 Solution proposée : Architecture de défense multi-couche

Pour répondre efficacement aux menaces identifiées, nous proposons une architecture de sécurité en profondeur (Defense in Depth) basée sur des solutions Open-Source éprouvées. Cette approche permet de filtrer le trafic à différents niveaux du modèle OSI, garantissant ainsi une protection robuste tant au niveau réseau qu'applicatif.

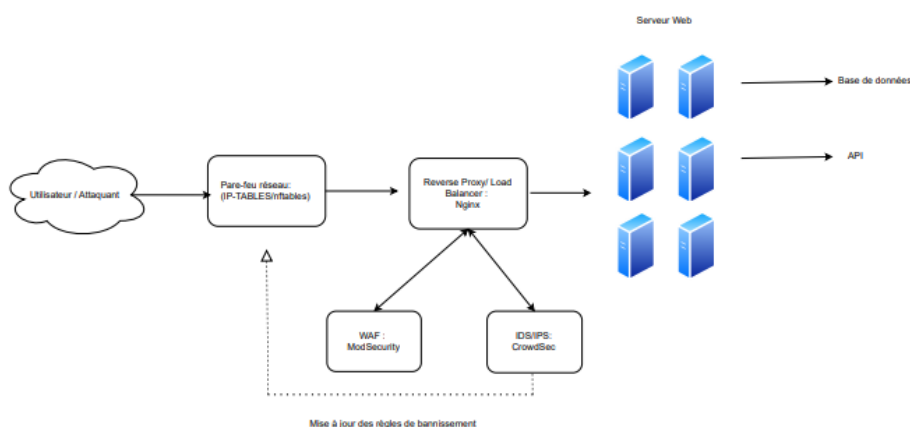


FIGURE 1.1 – Architecture de défense proposée intégrant Nginx, ModSecurity et CrowdSec

1.7.1 Composants techniques de la solution

L'architecture s'articule autour de quatre piliers technologiques majeurs qui collaborent pour sécuriser les serveurs backend :

- **Pare-feu réseau (IP-TABLES/nftables)** : Il constitue la première ligne de défense au niveau des couches 3 et 4. Son rôle est de filtrer les paquets en fonction des adresses IP, des ports et des protocoles, bloquant ainsi les tentatives d'intrusion massives avant qu'elles n'atteignent l'application.

- **Reverse Proxy / Load Balancer (Nginx)** : Positionné comme point d'entrée unique, Nginx gère la terminaison SSL et distribue la charge entre les différents serveurs web. Il sert de pivot central pour l'inspection du trafic.
- **WAF (ModSecurity)** : Intégré directement à Nginx, ce Web Application Firewall est chargé de l'inspection profonde des requêtes HTTP au niveau de la couche 7. Il analyse les contenus pour détecter et bloquer les injections SQL, les failles XSS et les entités XML malveillantes (XXE).
- **IDS/IPS (CrowdSec)** : Ce système de détection et de prévention d'intrusion analyse les journaux en temps réel pour identifier des comportements suspects. Contrairement au WAF qui analyse le contenu, CrowdSec se concentre sur la réputation et le comportement des adresses IP.

1.7.2 Mécanisme de remédiation automatique

La force de cette architecture réside dans l'interaction entre CrowdSec et le pare-feu réseau. Lorsqu'une menace est détectée au niveau applicatif ou comportemental par CrowdSec, le système génère automatiquement une consigne de blocage.

Cette consigne est transmise au pare-feu réseau pour effectuer une mise à jour dynamique des règles de bannissement. Ainsi, l'attaquant est bloqué dès la périphérie du réseau, libérant les ressources du Reverse Proxy et des serveurs backend pour le trafic légitime.

1.7.3 Infrastructure Backend

Une fois le trafic filtré et validé, il est transmis à l'infrastructure interne qui se compose de :

- **Serveurs Web** : Plusieurs instances pour assurer la haute disponibilité des services.
- **API** : Pour le traitement des données et la communication entre services.
- **Bases de données** : Stockage sécurisé des informations, protégé contre les accès directs depuis l'extérieur.

Chapitre 2

Déploiement de l'Environnement de Test (Proof of Concept)

2.1 Introduction

Afin d'évaluer l'efficacité de l'architecture de défense proposée, nous avons conçu et déployé un environnement de laboratoire isolé (Sandbox). Cette approche de type *Proof of Concept* (PoC) permet de simuler des scénarios d'attaques réalistes tout en garantissant un confinement total des activités malveillantes hors de tout réseau de production.

2.2 Topologie de l'infrastructure réseau et virtuelle

Le laboratoire repose sur une architecture virtualisée stricte, gérée par l'hyperviseur **Oracle VM VirtualBox**. Il se compose de deux machines virtuelles (VM) distinctes aux rôles antagonistes : une machine attaquante (Red Team) et une machine cible défensive (Blue Team).

Pour assurer à la fois l'isolation des tests et l'accès aux ressources externes (mises à jour, téléchargement de paquets), chaque machine virtuelle est équipée de deux interfaces réseau :

- **Interface NAT (Network Address Translation)** : Permet aux machines d'accéder à Internet pour le téléchargement des dépendances et outils nécessaires au déploiement.
- **Interface Host-Only (Réseau privé hôte)** : Constitue le cœur du laboratoire. Elle permet une communication directe, bidirectionnelle et isolée entre la machine Kali et la machine Ubuntu (permettant notamment le routage et les requêtes ICMP de type ping).

2.2.1 La machine attaquante (Kali Linux)

Le rôle de l'attaquant est opéré par une distribution Kali Linux. Conçue spécifiquement pour les tests d'intrusion, cette machine intègre nativement l'ensemble des outils offensifs nécessaires à l'évaluation de notre architecture, notamment **Nmap** pour la reconnaissance réseau, **hping3** pour les attaques de type DDoS (SYN Flood), et **Nikto** ou **Burp Suite** pour les attaques applicatives de couche 7.

2.2.2 La machine cible et défensive (Ubuntu Server)

La seconde machine virtuelle fonctionne sous la distribution Ubuntu. Elle héberge l'application web vulnérable ainsi que l'ensemble de la pile de sécurité (Pare-feu, Reverse Proxy, WAF et IPS).

2.3 Déploiement conteneurisé de l'application et du Reverse Proxy

Afin de garantir une isolation optimale des services, une portabilité accrue et une gestion simplifiée des ressources, nous avons opté pour une approche orientée conteneurs en utilisant **Docker**.

Sur la machine hôte Ubuntu, deux conteneurs distincts ont été déployés :

- **Conteneur DVWA (Backend)** : Héberge l'application *Damn Vulnerable Web App* associée à sa base de données. Ce conteneur écoute sur un port interne spécifique (ex : 8080) et n'est pas exposé directement à l'extérieur. L'utilisation de DVWA est primordiale car elle offre un environnement intentionnellement vulnérable, idéal pour tester les injections SQL et les failles XSS.
- **Conteneur Nginx (Reverse Proxy)** : Positionné en frontal, ce conteneur intercepte l'ensemble du trafic entrant sur le port HTTP standard (80). Il est configuré pour relayer (proxy_pass) le trafic légitime vers le conteneur DVWA. C'est la séparation stricte de ces deux entités qui fonde la robustesse de l'architecture.

2.4 Implémentation des couches de défense

Le déploiement des mécanismes de sécurité s'effectue à plusieurs niveaux du modèle OSI, conformément au principe de défense en profondeur théorisé dans le chapitre précédent.

2.4.1 Filtrage réseau bas niveau (iptables)

Au niveau de la couche réseau de la machine hôte Ubuntu, des règles **iptables** strictes ont été configurées. La politique par défaut a été définie sur **DROP**, bloquant silencieusement tout trafic non autorisé. Des règles spécifiques ont été intégrées pour :

- Rejeter les paquets malformés (scans XMAS, NULL).
- Imposer une limite de taux (Rate Limiting) sur les nouvelles connexions TCP (SYN), atténuant ainsi l'impact des attaques par inondation de type SYN Flood.

2.4.2 Intégration du WAF ModSecurity

Le pare-feu applicatif ModSecurity a été compilé et intégré directement au service Nginx. Pour contrer les attaques génériques (injections SQL, XSS), le jeu de règles standard de l'industrie, *OWASP Core Rule Set (CRS)*, a été activé en mode blocage (SecRuleEngine On).

Afin d'illustrer la capacité de réaliser du *Virtual Patching*, une règle personnalisée a été rédigée pour interdire formellement l'accès au fichier de configuration critique de

l'application (`setup.php`), renvoyant un code HTTP 403 Forbidden à tout attaquant tentant d'y accéder.

2.4.3 Déploiement de l'IPS comportemental CrowdSec

Pour la détection dynamique, l'agent CrowdSec a été installé sur l'hôte Ubuntu. Il a été configuré pour ingérer et analyser en temps réel les journaux d'accès et d'erreurs générés par le Reverse Proxy Nginx.

Ajustement de l'environnement : Dans sa configuration par défaut, CrowdSec place les adresses IP privées (RFC 1918) sur liste blanche. Dans le cadre de notre laboratoire (réseau Host-Only en `192.168.56.x`), il a été nécessaire de désactiver cette exclusion locale (`whitelists.yaml`) afin que le moteur puisse effectivement sanctionner la machine Kali lors des attaques.

Enfin, le composant *Firewall Bouncer* a été connecté à CrowdSec, lui permettant d'ajouter dynamiquement l'adresse IP de l'attaquant dans les règles de blocage `iptables` suite à la détection d'un comportement malveillant (comme un scan de vulnérabilité intensif).

Chapitre 3

Durcissement du Réseau et Attaques Volumétriques (Couche 4)

3.1 Introduction

Ce chapitre présente la première phase de concrétisation technique de notre architecture de défense en profondeur. Après avoir mis en place l’environnement de laboratoire, nous détaillons ici l’implémentation effective des protections de la couche réseau (Couche 4 du modèle OSI). Nous présenterons le traitement d’un flux TCP hostile, de l’optimisation de la mémoire du noyau Linux jusqu’à la limitation dynamique du débit par adresse IP, visant à bloquer les attaques volumétriques massives avant qu’elles n’atteignent le serveur applicatif.

3.2 Contexte et Analyse de la Menace : Le SYN Flood

Avant de protéger la couche applicative, la première ligne de défense doit éliminer le trafic réseau indésirable visant à saturer l’infrastructure. L’une des menaces les plus critiques à ce niveau est l’attaque **SYN Flood**.

Dans une connexion TCP normale, le processus d’établissement (Three-Way Handshake) requiert un échange de paquets précis (SYN, SYN-ACK, ACK). Lors d’une attaque SYN Flood, l’attaquant envoie massivement des requêtes d’initialisation (SYN) sans jamais finaliser la connexion.

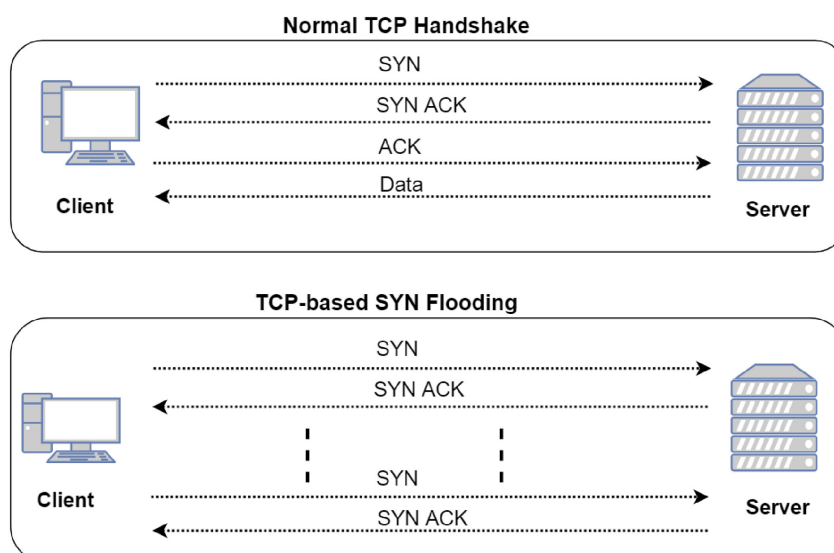


FIGURE 3.1 – Mécanisme d'épuisement des ressources via SYN Flood

L'impact direct est l'**épuisement du Backlog** : le serveur cible alloue de la mémoire (RAM) pour chaque connexion semi-ouverte en attente. Une fois cette file d'attente saturée, le serveur devient totalement incapable d'accepter de nouvelles connexions légitimes, provoquant un déni de service (DDoS).

3.3 Implémentation des Défenses Réseau

3.3.1 Durcissement du Noyau Linux (Sysctl)

Pour protéger le noyau contre cet épuisement de mémoire, la première étape consiste à ajuster les paramètres de gestion TCP via le fichier de configuration du système.

La solution principale réside dans l'activation des **SYN Cookies**. Ce mécanisme permet au serveur de valider mathématiquement une connexion entrante en générant un "cookie" cryptographique, éliminant ainsi le besoin de stocker l'état de la connexion dans la RAM avant la réponse finale du client. (*Le détail des paramètres configurés dans `sysctl.conf` est disponible en Annexe A.1.1*).

3.3.2 Limitation de Débit Dynamique (Iptables Hashlimit)

Si les SYN Cookies protègent la mémoire, ils ne limitent pas la bande passante consommée. Pour contrer un attaquant spécifique sans impacter le trafic global, nous avons déployé le pare-feu `iptables` avec le module avancé `hashlimit`.

Contrairement à une limitation stricte qui bloquerait tous les utilisateurs de l'application, `hashlimit` crée une table de hachage dynamique générant un **compteur de débit indépendant pour chaque adresse IP source**. (*Le script complet de configuration des règles de filtrage est disponible en Annexe A.1.2*).

Les paramètres clés de notre implémentation garantissent une protection granulaire :

- **Identification granulaire** (`-hashlimit-mode srcip`) : L'attaquant sature son propre quota (bucket) sans affecter les requêtes des autres clients.

- **Seuil de tolérance** (-hashlimit 30/sec) : Le débit est plafonné à 30 requêtes par seconde, ce qui est largement suffisant pour une navigation web humaine.
- **Flexibilité** (-hashlimit-burst 40) : Tolérance accordée pour un pic initial de connexions (utile lors du chargement simultané d'images sur une page web).

3.4 Validation et Preuves de Concept

Pour valider l'efficacité de cette architecture, des attaques SYN Flood ont été simulées depuis la machine attaquante vers l'infrastructure protégée à l'aide de l'outil `hping3`.

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
└─$ sudo hping3 -S --flood -V -p 80 192.168.56.20
[sudo] password for kali:
using eth1, addr: 192.168.56.10, MTU: 1500
HPING 192.168.56.20 (eth1 192.168.56.20): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown

Every 1.0s: sudo iptables -nvL --line-numbers
Chain INPUT (policy ACCEPT 2826K packets, 116M bytes)
num  pkts bytes target    prot opt in     out     source            destination
1    4618K 185M SYN_FLOOD 6  --  *      *        0.0.0.0/0         0.0.0.0/0         tcp flags:0x17/0x02

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source            destination

Chain CROWDSEC_CHAIN (0 references)
num  pkts bytes target    prot opt in     out     source            destination

Chain SYN_FLOOD (1 references)
num  pkts bytes target    prot opt in     out     source            destination
1    24304 972K RETURN 0  --  *      *        0.0.0.0/0         0.0.0.0/0
2    4594K 184M DROP 0  --  *      *        0.0.0.0/0         0.0.0.0/0

```

FIGURE 3.2 – Interception en temps réel des paquets hostiles par Iptables

Le tableau ci-dessous synthétise les performances observées :

Configuration	Débit de l'attaque	Résultat Système	Impact Utilisateur Légitime
Sans protection	1000+ paquets/s	Épuisement RAM (Backlog)	Déni de service total (Timeout)
Sysctl uniquement	1000+ paquets/s	RAM préservée	Ralentissements réseau mineurs
Sysctl + Hashlimit	Bloqué à 30 req/s	Paquets excédentaires DROP	Navigation fluide et instantanée

TABLE 3.1 – Comparatif des performances sous attaque volumétrique

3.5 Conclusion

La phase de réalisation a permis de transformer les exigences théoriques de protection réseau en une solution technique concrète. L'intégration combinée des SYN Cookies au niveau du noyau et du module `hashlimit` d'Iptables offre à l'infrastructure une résilience significative face aux attaques par saturation volumétrique. Les tests effectués confirment que ces mécanismes de défense de première ligne réagissent correctement en isolant silencieusement le trafic malveillant (Drop), tout en maintenant une haute disponibilité du service pour les utilisateurs légitimes. Le trafic réseau étant désormais assaini, nous pouvons procéder à l'analyse de la couche applicative.

Chapitre 4

Protection Applicative et Volumétrique (Couche 7)

4.1 Introduction

Ce chapitre présente la deuxième phase technique de notre architecture de sécurité. Après avoir consolidé la couche réseau, nous détaillons ici la protection de la couche applicative (Couche 7 du modèle OSI). Nous présenterons les mécanismes mis en œuvre pour contrer les menaces plus insidieuses qui imitent le trafic HTTP légitime. L'objectif de cette étape est double : mitiger les attaques par épuisement de ressources (comme Slowloris) en optimisant le Reverse Proxy Nginx, et déployer un pare-feu applicatif (WAF) pour bloquer les tentatives d'injections (SQLi, XSS) avant qu'elles n'atteignent le serveur backend.

4.2 Mitigation Volumétrique : L'Attaque Slowloris

4.2.1 Contexte et Analyse de la Menace

Une fois les attaques réseau de force brute (SYN Flood) bloquées, le trafic malveillant tente de saturer le serveur au niveau de l'application. L'attaque **Slowloris** est emblématique de cette approche : c'est un déni de service de type *Low-and-Slow*.

Plutôt que de submerger la bande passante, l'attaquant ouvre des milliers de connexions simultanées et envoie les en-têtes HTTP de manière extrêmement lente et fragmentée.

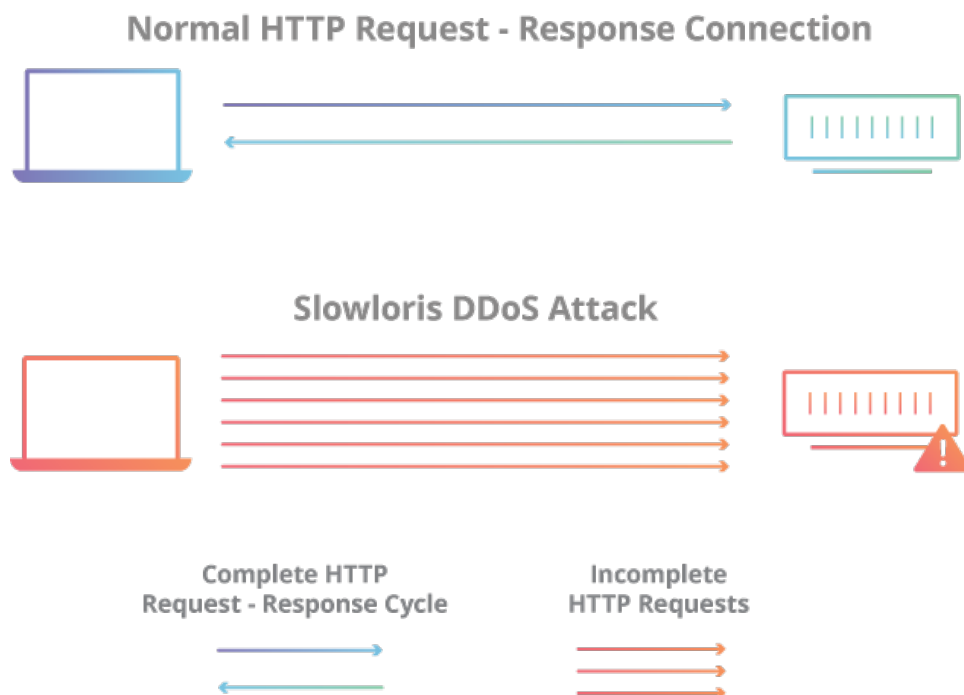


FIGURE 4.1 – Mécanisme de saturation des ressources applicatives par Slowloris

Face à cette menace, le choix du serveur web frontal est crucial. Un serveur classique comme Apache (modèle *Thread-Based*) alloue un processus lourd par connexion et s'épuise rapidement. À l'inverse, nous avons opté pour **Nginx**, dont l'architecture asynchrone (*Event-Driven*) permet de gérer des milliers de connexions simultanées sans gaspiller la mémoire.

4.2.2 Durcissement de Nginx (Timeouts et Rate Limiting)

Pour neutraliser définitivement Slowloris, nous avons configuré Nginx pour être **intolérant** envers les clients anormalement lents.

La configuration repose sur des **délais d'expiration (timeouts) agressifs**. Si un client (l'attaquant) ne parvient pas à transmettre l'intégralité de son en-tête HTTP dans un délai très court (10 secondes), Nginx clôture activement la connexion avec une erreur **408 Request Time-out**, libérant ainsi instantanément la ressource. De plus, une zone de mémoire a été définie pour limiter le taux de requêtes par IP. (*Les paramètres de configuration appliqués dans `nginx.conf` sont détaillés en Annexe A.2.1*).

```

(kali@kali)-[~]
└─$ sudo slowhttptest -c 500 -H -g -o slohttp -i 10 -r 200 -t GET
-u http://192.168.56.20/DVWA/ -x 24 -p 3

slowhttptest version 1.9.0
- https://github.com/shekyan/slowhttptest -
test type:                SLOW HEADERS
number of connections:    500
URL:                      http://192.168.56.20/DVWA/
verb:                     GET
cookie:
Content-Length header value: 4096
follow up data max size:  52
interval between follow up data: 10 seconds
connections per seconds:  200
probe connection timeout:  3 seconds
test duration:            240 seconds
using proxy:              no proxy

Wed Jan  7 16:21:48 2026:
slow HTTP test status on 20th second:
initializing:      0
pending:           38
connected:         157
error:             0
closed:            305
service available: YES

```

FIGURE 4.2 – Échec de l’attaque Slowloris face à l’architecture durcie de Nginx

4.3 Filtrage Applicatif : WAF ModSecurity

4.3.1 Principes d’Inspection Profonde

Pour contrer les attaques ciblant directement le code de l’application (DVWA), nous avons intégré le **Web Application Firewall (WAF) ModSecurity** directement au Reverse Proxy.

Le WAF agit comme une douane applicative. Il effectue une *inspection profonde des paquets* (Deep Packet Inspection) en analysant le contenu exact des requêtes HTTP (méthodes GET, POST, Cookies). Il s’appuie sur le jeu de règles standard de l’industrie : l’**OWASP Core Rule Set (CRS) v3**. (La configuration d’activation du WAF sur le virtual host est disponible en Annexe A.2.2).

La force de ModSecurity réside dans sa capacité de **normalisation** : même si l’attaquant tente d’obfusquer son code (par exemple via un encodage URL tel que %3Cscript%3E), le WAF le décrypte systématiquement avant de le comparer à ses signatures virales.

4.3.2 Neutralisation des Injections (SQLi et XSS)

- **Cross-Site Scripting (XSS)** : L’injection de code JavaScript malveillant est interceptée par les règles d’escalade de protocole de l’OWASP, qui repèrent les balises

`<script>` ou les événements `onload` cachés dans les paramètres.

- **Injection SQL (SQLi)** : La manipulation des bases de données est bloquée par l'identification de caractères d'échappement et de mots-clés SQL critiques (`UNION`, `DROP`, `-`, `'`).

4.3.3 Restriction d'accès aux pages sensibles

Au-delà du filtrage des charges malveillantes, une mesure de durcissement complémentaire a été mise en place afin de restreindre l'accès à certaines ressources sensibles de l'application. L'objectif est d'empêcher l'exposition de d'administration, d'initialisation ou de maintenance qui ne doivent pas être accessibles en exploitation normale.

Dans notre environnement de test, cette politique a notamment été appliquée à la page `setup.php` de DVWA. Cette ressource permet des opérations de configuration sensibles et ne doit donc pas être accessible librement depuis le réseau. Une règle dédiée au niveau du Reverse Proxy Nginx interdit son accès direct et retourne une erreur `403 Forbidden`.

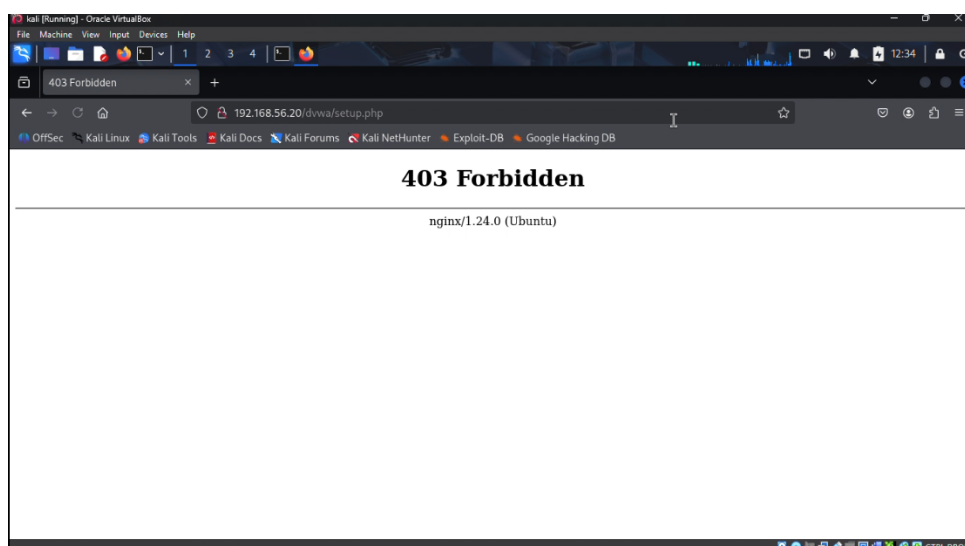


FIGURE 4.3 – Échec de l'accès à la page `setup.php`

Cette mesure présente plusieurs avantages : elle réduit la surface d'attaque, empêche la réinitialisation ou la reconfiguration non autorisée de l'application, et complète l'approche de défense en profondeur déjà assurée par le WAF ModSecurity. Ainsi, la sécurité ne repose pas uniquement sur la détection des requêtes malveillantes, mais aussi sur la limitation stricte des ressources exposées.

4.4 Résultats et Preuves de Concept

Les tests de pénétration ont été relancés (sqli et xss via le navigateur) sur l'infrastructure désormais protégée par le WAF.

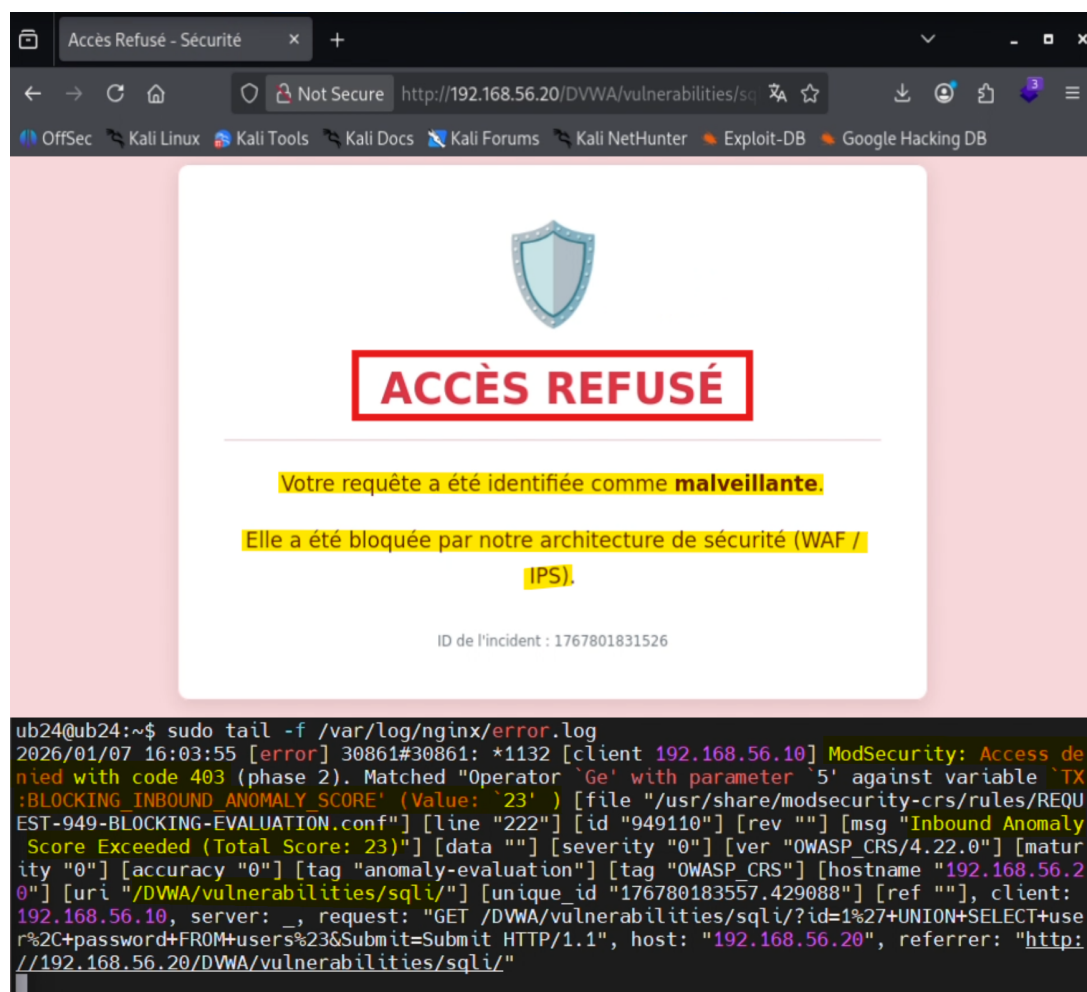


FIGURE 4.4 – Interception immédiate d’une charge utile malveillante par le WAF

Dès qu’une signature hostile est détectée, le WAF rompt la communication avant même d’interroger la base de données. Le client reçoit une erreur **HTTP 403 Forbidden**, et l’événement est consigné sous la mention "Access Denied" dans les journaux de sécurité.

Type d’Attaque	Outil utilisé	Réponse du Serveur	Statut de la Menace
Injection SQL	Navigateur web	Erreur 403 Forbidden	Bloqué (WAF)
XSS Réfléchi	Navigateur web	Erreur 403 Forbidden	Bloqué (WAF)
Navigation Standard	Navigateur web	Code 200 OK	Autorisé

TABLE 4.1 – Bilan des interceptions applicatives par ModSecurity

4.5 Conclusion

La phase de réalisation de la couche 7 a permis de transformer Nginx en un véritable rempart applicatif. L'optimisation des délais d'expiration (timeouts) garantit une résilience totale face à l'épuisement des ressources (Slowloris), tandis que l'intégration du WAF ModSecurity assure le filtrage strict des données entrantes. Les tests de validation confirment que les injections critiques (SQLi, XSS) sont interceptées systématiquement, préservant ainsi l'intégrité de la base de données et la sécurité des utilisateurs. Néanmoins, bien qu'efficace, cette défense reste passive : elle bloque la requête mais laisse l'attaquant libre de réessayer. Le chapitre suivant s'attachera à résoudre cette limite en déployant une réponse comportementale active.

Chapitre 5

Réponse Active (IPS) et Supervision

5.1 Introduction

Ce chapitre présente la dernière phase technique de notre architecture de défense en profondeur. Après avoir sécurisé les couches réseau et applicative par des mesures préventives, nous détaillons ici l'implémentation d'une réponse active et d'une supervision centralisée. Nous présenterons le cycle de traitement d'une menace, de sa détection comportementale par l'analyse continue des journaux du WAF, jusqu'à l'application automatique d'un bannissement réseau (IPS). Enfin, nous détaillerons la mise en place d'un tableau de bord (Dashboard SOC) permettant la visualisation de ces événements et la gestion proactive de la sécurité.

5.2 Contexte et Enjeux de la Sécurité Active

Une fois la couche applicative protégée par le WAF (ModSecurity), le trafic malveillant est bloqué requête par requête (Erreur 403). Cependant, une défense purement passive présente une limite : l'attaquant peut continuer à solliciter le serveur indéfiniment, gaspillant ainsi de la bande passante et des ressources de calcul.

Le concept de *Defense in Depth* impose d'ajouter une couche **IPS (Intrusion Prevention System)** capable de :

- **Analyser le comportement** global d'une connexion (fréquence des erreurs 403, tentatives de scan d'arborescence, force brute).
- **Prendre des décisions automatiques** basées sur un seuil de tolérance.
- **Mettre à jour dynamiquement** les règles du pare-feu réseau pour isoler l'attaquant à la source.

5.3 Déploiement de l'IPS : CrowdSec

5.3.1 Architecture

CrowdSec repose sur une architecture moderne et modulaire :

- **Découplage** : L'Agent (qui détecte) est séparé du Bouncer (qui sanctionne).
- **Intégration WAF** : Grâce à des collections spécifiques, CrowdSec comprend nativement les journaux **Access Denied** générés par ModSecurity.

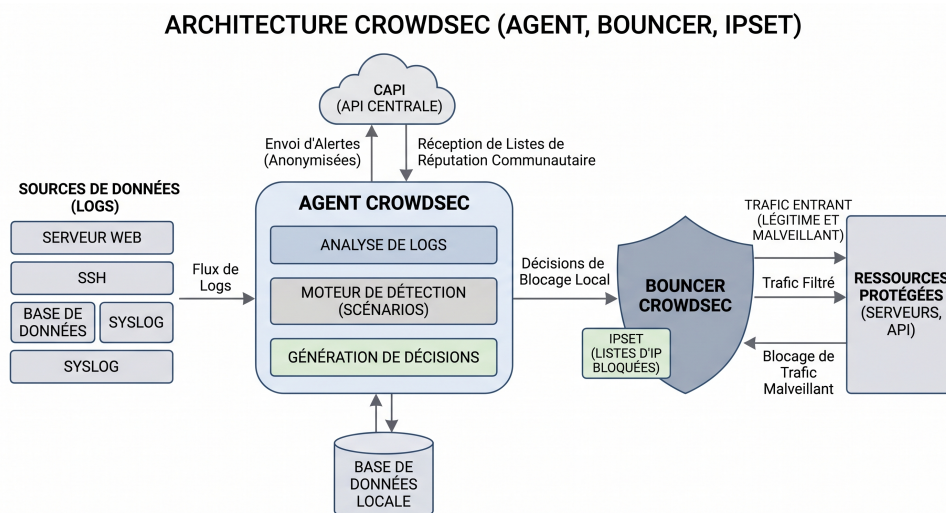


FIGURE 5.1 – Architecture Agent/Bouncer et flux de décision de CrowdSec

5.3.2 Mécanisme de Détection (Leaky Bucket)

Le moteur de détection de CrowdSec s'appuie sur le principe du **Leaky Bucket** (Seau Percé) : Chaque erreur générée par un utilisateur remplit un "seau" virtuel alloué à son adresse IP. Si les erreurs s'accumulent plus vite que le seau ne se vide (indiquant par exemple un scan de vulnérabilités via un outil comme Nikto), le seau déborde. Ce débordement déclenche instantanément un scénario de remédiation. *(Le détail des commandes d'installation et de liaison avec le WAF est fourni en Annexe A.3.1).*

5.3.3 Remédiation Réseau via IPSet

La force de l'IPS réside dans son intégration avec le noyau Linux. Au lieu d'ajouter des milliers de règles `iptables` individuelles (ce qui ralentirait le routage légitime), le Bouncer CrowdSec utilise **IPSet**. Il s'agit d'une structure de données optimisée en mémoire permettant de vérifier instantanément si une IP entrante figure sur la liste des entités bannies, avant même qu'elle ne traverse la pile TCP/IP.

5.4 Supervision et Pilotage (SOC) avec Metabase

Transformer les journaux techniques (logs) en indicateurs de performance (KPIs) est essentiel pour le pilotage d'un système de sécurité. Pour ce faire, nous avons déployé la plateforme **Metabase** via des conteneurs Docker. *(Le fichier de déploiement Docker-Compose est disponible en Annexe A.3.2).*

Metabase est connecté en *lecture seule* à la base de données de CrowdSec. Grâce à des requêtes SQL natives créées spécifiquement pour ce projet *(Voir Annexe A.3.3)*, nous avons pu concevoir un tableau de bord affichant :

- Le volume d'attaquants uniques identifiés.
- La traduction des scénarios techniques en langage clair (ex : "Injection SQL" au lieu de "REQUEST-942").
- Un journal des menaces en temps réel détaillant la source et la fréquence des attaques.

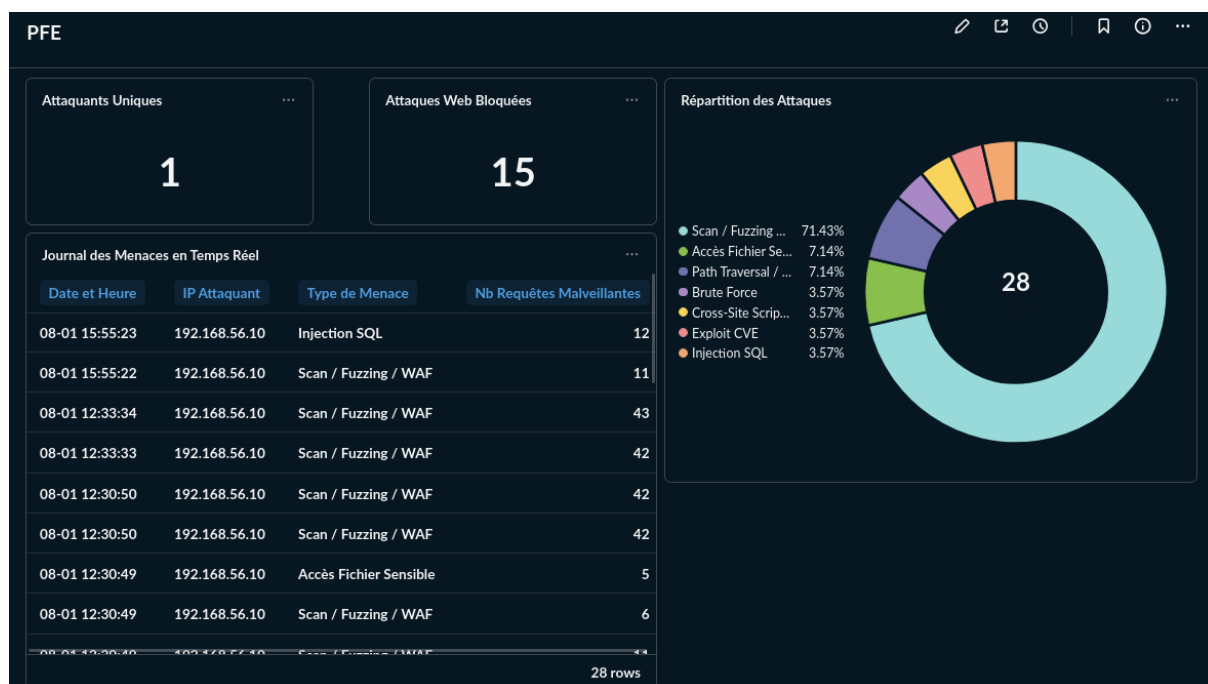


FIGURE 5.2 – Tableau de bord SOC pour la supervision en temps réel

5.5 Tests et Validation de l'IPS

Pour valider la chaîne complète, des attaques simulant des comportements agressifs (scans de vulnérabilités via l'outil Nikto) ont été lancées depuis la machine attaquante.

```
(kali@kali)-[~]
$ nikto -h http://192.168.56.20/
- Nikto v2.5.0

+ Target IP: 192.168.56.20
+ Target Hostname: 192.168.56.20
+ Target Port: 80
+ Start Time: 2026-01-08 13:32:32 (GMT1)

+ Server: nginx/1.24.0 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See:
https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options

ub24@ub24:~$ sudo cscli decision list
```

ID	Source	Scope:Value	Reason	Action	Country
AS	Events	expiration Alert ID			
26	crowdsec 43	Ip:192.168.56.10 3h59m48s 26	crowdsecurity/http-crawl-non_statics	ban	

```
1 duplicated entries skipped
ub24@ub24:~$
```

FIGURE 5.3 – Constat du bannissement effectif de l'attaquant au niveau du pare-feu

Le tableau ci-dessous résume le comportement de la solution face à différentes sollicitations :

Comportement (Source)	Détection (Log WAF/Proxy)	Décision de l'IPS (CrowdSec)	Statut Final
Scan de vulnérabilités (Nikto)	Multiples Erreurs 403	Seuil dépassé → Ban de 4 heures	Bloqué (L3)
Épuisement de ressources (Slowloris)	Multiples Erreurs 408	Scénario HTTP déclenché → Ban	Bloqué (L3)
Navigation légitime	Codes 200 OK	Seau vide (Aucune alerte)	Autorisé

TABLE 5.1 – Synthèse des réponses actives du système CrowdSec

5.6 Conclusion

La phase de réalisation a permis de transformer une architecture de défense jusqu'alors purement passive en un écosystème proactif. L'intégration combinée de l'Agent CrowdSec, du pare-feu dynamique via IPSet, et de la plateforme de supervision Metabase offre au système une résilience significative face aux attaques automatisées et persistantes. Les tests effectués confirment que les mécanismes de détection comportementale réagissent avec précision aux menaces simulées : les attaquants sont isolés au niveau réseau dès que le seuil de tolérance est franchi, libérant ainsi les ressources du WAF. En parallèle, le tableau de bord SOC remplit parfaitement son rôle en traduisant ces actions techniques en indicateurs décisionnels clairs.

Conclusion Générale

Dans le cadre de ce PFE, nous avons conçu et implémenté une architecture de sécurité en profondeur (*Defense in Depth*) destinée à protéger une infrastructure web vulnérable. Cette solution permet une sécurisation robuste et multicouche, garantissant la disponibilité, l'intégrité et la confidentialité des services sans nécessiter de modification du code source de l'application cible. Le système inclut des mécanismes de défense modernes tels que le durcissement du noyau Linux, un pare-feu applicatif (WAF ModSecurity), un système de prévention d'intrusion comportemental (IPS CrowdSec), tout en offrant une interface de supervision centralisée (SOC) pour l'analyse des menaces en temps réel.

Ce rapport a couvert l'ensemble des étapes du projet, depuis l'identification des menaces critiques (attaques volumétriques, Top 10 OWASP) et l'analyse des besoins de sécurité, jusqu'à l'implémentation technique des barrières de protection aux couches réseau et applicative. Nous avons mis en place une méthodologie de travail rigoureuse, validée par des tests de pénétration simulant des attaques réelles (SYN Flood, Slowloris, Injections SQL et XSS). Par ailleurs, nous avons détaillé l'écosystème technologique déployé pour durcir l'infrastructure, reposant principalement sur Iptables, Nginx, Docker et des outils de sécurité open-source de pointe.

La réalisation de ce bouclier de sécurité a été une expérience extrêmement enrichissante sur le plan technique pour notre groupe. Elle nous a permis d'approfondir notre maîtrise des standards de cybersécurité, de l'administration système et de la conception d'architectures résilientes. Nous avons particulièrement renforcé nos compétences en analyse de trafic réseau, en configuration de filtres avancés (WAF), en remédiation automatique et en centralisation d'événements de sécurité.

Pour l'avenir, et afin de transposer ce laboratoire vers un environnement de production à grande échelle, plusieurs axes d'amélioration peuvent être envisagés. Nous pourrions, par exemple, intégrer le standard SSL/TLS via Let's Encrypt pour chiffrer le trafic de bout en bout, ou encore automatiser le déploiement complet de cette architecture via des outils d'*Infrastructure as Code* comme Ansible. De plus, la migration de la base de données locale vers une pile de gestion centralisée des logs (comme ELK ou Wazuh) permettrait de gérer des volumes d'alertes plus importants et d'assurer une corrélation d'événements multi-sites.

En somme, ce projet constitue une réponse concrète et efficace pour la protection du patrimoine numérique face aux cybermenaces modernes. En neutralisant avec succès les scénarios d'attaques tout en préservant l'expérience des utilisateurs légitimes, cette architecture fortifiée prouve que la sécurité est un processus d'ingénierie continu. Nous sommes convaincus que les compétences acquises au cours de ce projet apporteront une valeur ajoutée significative dans nos futures carrières en cybersécurité.

Annexes Techniques

A.1 Scripts de Durcissement Réseau (Couche 4)

A.1.1 Configuration Sysctl (Protection Mémoire Kernel)

Extrait du fichier de configuration `/etc/sysctl.conf` visant à activer les mécanismes de protection contre l'épuisement de la file d'attente TCP.

```
# Activation des SYN Cookies (pas de stockage en RAM des connexions semi-ouvertes)
net.ipv4.tcp_syncookies = 1

# Augmentation de la taille de la file d'attente (Backlog)
net.ipv4.tcp_max_syn_backlog = 2048

# Réduction du nombre d'essais avant d'abandonner une connexion morte
net.ipv4.tcp_synack_retries = 2
```

A.1.2 Script Iptables (Limitation de débit dynamique)

Commandes exécutées pour configurer le pare-feu Netfilter, permettant d'isoler une adresse IP dépassant les quotas autorisés sans impacter le reste du réseau.

```
# 1. Création d'une chaîne dédiée pour le filtrage
sudo iptables -N SYN_FLOOD

# 2. Redirection des paquets TCP (flag SYN) vers cette nouvelle chaîne
sudo iptables -A INPUT -p tcp --syn -j SYN_FLOOD

# 3. Application de la règle Hashlimit par adresse IP source
sudo iptables -A SYN_FLOOD -m hashlimit \
--hashlimit-name syn_protection \
--hashlimit-mode srcip \
--hashlimit 30/sec \
--hashlimit-burst 40 \
-j RETURN

# 4. Rejet silencieux des paquets excédant la limite
sudo iptables -A SYN_FLOOD -j DROP
```

A.2 Configurations de Sécurité Applicative (Couche 7)

A.2.1 Durcissement Nginx (Anti-Slowloris et Rate Limiting)

Extrait du fichier de configuration global `/etc/nginx/nginx.conf` (dans le bloc `http`). Ces paramètres visent à rejeter les connexions anormalement lentes pour contrer les attaques par épuisement de ressources.

```
# Création d'une zone de mémoire pour limiter le taux de requêtes par IP
limit_req_zone $binary_remote_addr zone=one:10m rate=10r/s;

# Timeouts agressifs : clôture la connexion si le client est trop lent
# (Arme principale contre le maintien des en-têtes par Slowloris)
client_header_timeout 10s;
client_body_timeout 10s;

# Réduction du temps de maintien d'une connexion inactive
keepalive_timeout 5s;
```

A.2.2 Activation du WAF ModSecurity sur le Virtual Host

Extrait du fichier de configuration du proxy inverse (ex : `/etc/nginx/conf.d/reverse-proxy.conf`). Cette configuration permet d'activer le moteur d'inspection profonde avant de relayer le trafic vers l'application vulnérable.

```
server {
    listen 80;
    server_name 192.168.56.20;

    location / {
        # Activation du moteur Web Application Firewall
        modsecurity on;

        # Chargement du fichier principal incluant les règles OWASP CRS
        modsecurity_rules_file /etc/nginx/modsec/main.conf;

        # Relais du trafic filtré et légitime vers le backend (DVWA)
        proxy_pass http://192.168.56.30;
    }
}
```

A.3 Configurations de la Réponse Active et Supervision (Couche IPS)

A.3.1 Déploiement et liaison de CrowdSec avec le WAF

Commandes utilisées pour installer le moteur IPS, le module de remédiation Iptables (Bouncer), et la collection permettant la compréhension des journaux d'erreurs générés par ModSecurity.

```
# Mise à jour et installation des paquets
sudo apt update
sudo apt install crowdsec crowdsec-firewall-bouncer-iptables

# Pont entre WAF et IPS : Installation de la collection ModSecurity
sudo cscli collections install crowdsecurity/modsecurity

# Activation et démarrage du service
sudo systemctl enable crowdsec
sudo systemctl start crowdsec
```

Configuration de la Whitelist (Environnement de Laboratoire) Extrait du fichier `/etc/crowdsec/parsers/s02-enrich/whitelists.yaml`. Dans le cadre de ce laboratoire (Réseau privé 192.168.56.x), les exclusions par défaut des réseaux LAN ont dû être désactivées pour permettre le bannissement de la machine attaquante.

```
name: crowdsecurity/whitelists
whitelist:
  cidr:
    - "127.0.0.0/8"
    # Les réseaux suivants sont commentés pour les besoins du PoC :
    # - "10.0.0.0/8"
    # - "192.168.0.0/16"
```

A.3.2 Déploiement de Metabase (Docker Compose)

Fichier `docker-compose.yml` utilisé pour déployer le tableau de bord SOC. Il monte la base de données d'alertes de CrowdSec en lecture seule.

```
version: '3'
services:
  metabase:
    image: metabase/metabase:latest
    container_name: soc_dashboard
    ports:
      - "3000:3000"
    volumes:
      # Montage de la DB SQLite en lecture seule (:ro) pour éviter la corruption
      - /var/lib/crowdsec/data/crowdsec.db:/app/crowdsec.db:ro
    environment:
      - MB_DB_FILE=/app/metabase.db
```

A.3.3 Requêtes SQL Analytiques (Dashboard SOC)

Exemple de requêtes SQL natives créées dans Metabase pour générer les indicateurs clés (KPIs) en interrogeant la table `alerts` de CrowdSec.

```
-- Indicateur : Nombre d'attaquants uniques
SELECT count(DISTINCT source_ip) FROM alerts;
```

```
-- Indicateur : Durée moyenne de bannissement (Conversion en heures)
SELECT avg(duration) / 3600 FROM decisions WHERE type = 'ban';
```

```
-- Tableau : Journal des Menaces (Nettoyage et traduction des scénarios)
```

```
SELECT
    strftime('%d-%m %H:%M:%S', created_at) as "Date",
    source_ip as "IP Attaquant",
    CASE
        WHEN scenario LIKE '%sqli%' THEN 'Injection SQL'
        WHEN scenario LIKE '%xss%' THEN 'XSS'
        WHEN scenario LIKE '%path-traversal%' THEN 'Path Traversal'
        WHEN scenario LIKE '%bf%' OR scenario LIKE '%bruteforce%' THEN 'Brute Force'
        WHEN scenario LIKE '%probing%' OR scenario LIKE '%crawl%' THEN 'Scan / Fuzzing'
        ELSE scenario
    END as "Type de Menace",
    events_count as "Nb Requêtes"
FROM alerts
ORDER BY created_at DESC
LIMIT 50;
```

Bibliographie

- [1] OWASP, “Owasp top 10 - 2021 : The ten most critical web application security risks.” <https://owasp.org/www-project-top-ten/>, 2021. [Consulté le 15 Novembre 2025].
- [2] OWASP, “Owasp modsecurity core rule set (crs).” <https://coreruleset.org/>, 2025. [Consulté le 02 Décembre 2025].
- [3] T. SpiderLabs, “Modsecurity : Open source web application firewall.” <https://github.com/owasp-modsecurity/ModSecurity>, 2024. [Consulté le 10 Décembre 2025].
- [4] CrowdSec, “Crowdsec documentation : Collaborative behavior detection engine.” <https://docs.crowdsec.net/>, 2025. [Consulté le 18 Janvier 2026].
- [5] N. Inc., “Mitigating ddos attacks with nginx and nginx plus.” <https://www.nginx.com/blog/mitigating-ddos-attacks-with-nginx-and-nginx-plus/>, 2020. [Consulté le 25 Novembre 2025].
- [6] Cloudflare, “What is a syn flood attack? | cloudflare learning.” <https://www.cloudflare.com/learning/ddos/syn-flood-ddos-attack/>, 2025. [Consulté le 05 Janvier 2026].
- [7] N. C. Team, “iptables-extensions : Hashlimit module documentation.” <https://ipset.netfilter.org/iptables-extensions.man.html>, 2023. [Consulté le 12 Février 2026].
- [8] Metabase, “Metabase documentation : Business intelligence and analytics.” <https://www.metabase.com/docs/latest/>, 2025. [Consulté le 28 Février 2026].
- [9] R. Dewhurst, “Damn vulnerable web app (dvwa).” <https://github.com/digininja/DVWA>, 2025. [Consulté le 08 Novembre 2025].
- [10] D. Inc., “Docker documentation : Containerization platform.” <https://docs.docker.com/>, 2025. [Consulté le 02 Mars 2026].